AI · DATA · ARCHITECTURE

탐색적 데이터분석 방법

동국대학교 IR성과분석팀 이현우

구글 코랩

구글 코랩이란?

- 구글 코랩은 클라우드 기반의 파이썬 실행 환경
- 컴퓨터 사양이 좋지 않아도 구글의 강력한 서버를 빌려 사용하므로 속도가 빠르고,
- 구글 계정만 있다면 스마트폰이나 태블릿에서도 내가 짠 코드를 확인할 수 있음

- 주요특징
  1. 별도의 설치나 설정이 전혀 필요 없음
  2. 작성한 코드는 구글 드라이브(Google Drive)에 자동 저장
  3. 무료 GPU/TPU 제공을 통해 고가의 하드웨어 없이 딥러닝 모델을 학습할 수 있음
  4. 여러 사용자가 동시에 같은 노트북을 편집하고 공유할 수 있음
  5. TensorFlow, PyTorch, scikit-learn 등 주요 AI 프레임워크를 지원

구글 코랩이란?

- 코드 자동완성
  - 내용 입력 후 Enter키를 누르면 코드가 자동 완성됨.
  - 잠시 기다리면 회색 글씨(Ghost text)로 추천 코드가 연하게 표시됨.
  - Tab 키를 누르면 해당 코드가 바로 입력됨

- 사용 가능한 AI기능
  - 코드생성, 코드설명, 코드변환

구글 코랩(Google Colab)

구분	Colab Free (무료)	Colab Pro ($9.99/월)	Colab Pro+ ($49.99/월)
AI 기능 접근성	기본 지원 (일부 지역 제한 및 할당량 제한)	우선 제공	최우선 제공
AI 코드 자동완성 (Inline)	제한적 지원 (사용량 많을 시 자동 일시 중단)	무제한 및 빠른 속도 제공	무제한 및 최우선 속도 제공
Data Science Agent / Gemini	일일 이용 횟수 제한 (할당량 소진 시 24시간 대기)	컴퓨팅 단위(Compute Units) 소진 전까지 여유롭게 사용	대용량 할당량 제공으로 헤비 유저에게 적합
GPU/TPU 컴퓨팅 자원	기본 T4 GPU (세션 끊김 자주 발생)	고성능 T4, V100 GPU 선택 가능	A100, V100 등 최고 사양 GPU 우선 할당
백그라운드 실행	미지원 (웹 브라우저 닫으면 종료)	미지원	지원 (브라우저를 닫아도 최대 24시간 유지)
시스템 RAM	약 12~13GB	최대 32GB (High-RAM)	최대 52GB (High-RAM)
VM 가상 디스크 용량	약 100GB (임시 공간)	최대 100~225GB	최대 225GB 이상
최대 연속 실행 시간	이론상 12시간 (유휴 시 30~90분 내 중단)	최대 24시간	24시간 이상 + 백그라운드 실행
제공 Compute Units	없음 (기본 자원 제공)	월 100 CU	월 500 CU

Compute Units : Colab에서 GPU·TPU 같은 고성능 컴퓨팅 자원을 사용할 때 소모되는 사용량 단위

Compute Units (컴퓨팅 유닛) 소진 및 차감 방식

- **단가 기준:** $9.99에 100 CU 구매 가능 (**1 CU = 약 $0.10**)
- **GPU 스펙별 실제 차감 비율 (추정치):**
  - **Standard (CPU 전용):** 약 0.1~0.2 CU/시간 (100 CU로 약 500시간 이상 사용)
  - **T4 GPU:** 약 **1.5~2.0 CU/시간** (100 CU로 **약 50~65시간** 사용 가능 / 시간당 약 $0.15~$0.20)
  - **V100 GPU:** 약 **5.0~5.5 CU/시간** (100 CU로 **약 18~20시간** 사용 가능 / 시간당 약 $0.50~$0.55)
  - **A100 GPU (고성능):** 약 **12.0~13.0 CU/시간** (100 CU로 **약 7~8시간** 사용 가능 / 시간당 약 $1.20~$1.30)

코드 실행 단축키

•Ctrl + Enter (Mac: Cmd + Enter): 현재 셀 실행
•Shift + Enter: 현재 셀 실행 후 다음 셀로 이동 (없으면 새 셀 생성)
•Alt + Enter (Mac: Option + Enter): 현재 셀 실행 후 아래에 새 셀 삽입

셀 관리 단축키

먼저 Ctrl + M (Mac: Cmd + M)을 누른 후 아래 키를 입력

•Ctrl + M → B: 아래에 새 코드 셀 삽입
•Ctrl + M → A: 위에 새 코드 셀 삽입
•Ctrl + M → D: 현재 셀 삭제 (2번 누름)
•Ctrl + M → Y: 코드 셀로 변환
•Ctrl + M → M: 마크다운 셀로 변환
•Ctrl + M → Z: 셀 삭제 실행 취소

기타 유용한 단축키

•Ctrl + S (Mac: Cmd + S): 노트북 저장
•Ctrl + / (Mac: Cmd + /): 주석 토글
•Ctrl + M → H: 단축키 도움말 표시
•Tab: 자동 완성
•Shift + Tab: 함수 도움말 표시

AI 및 프롬프트 단축키

• 화면의 Gemini 아이콘 클릭 : Gemini AI 열기
• AI 코드 자동완성: 코드 입력 중 제안 표시 → Tab으로 제안 수락

런타임 및 세션 제어 단축키

•**Ctrl + M → I**: 현재 실행 중인 코드 강제 중단 (Interrupt)
•**Ctrl + M → .**: 런타임 재시작 (Restart Runtime)
•**Ctrl + F9 (Mac: Cmd + F9)**: 전체 셀 처음부터 끝까지 일괄 실행
•**Ctrl + F8 (Mac: Cmd + F8)**: 현재 셀 이전의 모든 셀 실행

코드 편집 및 이동 단축키

•**Ctrl + M → K**: 선택한 셀을 위로 이동
•**Ctrl + M → J**: 선택한 셀을 아래로 이동
•**Ctrl + M → -[minus]**: 현재 커서 위치를 기준으로 셀 두 개로 나누기
•**Shift + M**: 선택된 여러 셀 하나로 합치기
•**Ctrl + D (Mac: Cmd + D)**: 동일한 단어/변수명 다중 선택 (한 번에 이름 변경할 때 유용)
•**Alt + Click (Mac: Option + Click)**: 멀티 커서 생성 (여러 줄 동시 수정)

화면 및 탐색 단축키

•**Ctrl + Shift + P**: 모든 메뉴 명령어를 검색해 실행할 수 있는 '명령 팔레트(Command Palette)' 열기
•**Ctrl + M → O**: 실행 결과 출력창 숨기기 / 다시 보이기

5분간

단축키 및 실습환경 활용

파이썬(python)

구분	Python	R	SAS
주요 용도	데이터분석, AI/ML, 자동화, 웹/백엔드	통계분석, 연구, 시각화	통계분석, 기업·금융·제약
장점	활용 범위가 매우 넓고 AI 생태계 강함	통계 분석에 강함	기업용 통계·관리 기능 강함
생태계	pandas, NumPy, scikit-learn, PyTorch, TensorFlow	tidyverse, ggplot2, caret	SAS 자체 생태계
비용	무료·오픈소스	무료·오픈소스	상용
2026 PYPL 점유율	**53.9%**	**3.87%**	PYPL 주요 순위에 포함되지 않음

1. 변수란 무엇인가?

- 변수는 프로그램에서 객체를 가리키도록 붙이는 이름
- 객체는 숫자·문자열·표처럼 프로그램에서 다루는 데이터나 기능

2. 왜 필요한가?

- 같은 값을 여러 곳에서 사용하거나 값이 바뀔 때, 이름을 통해 의미를 유지하며 다시 사용하기 위해

3. 코드

```
year = 2026
next_year = year + 1
print(year, next_year)
print(type(year).__name__)
year = "2026"
print(type(year).__name__)
```

2026 2027
int
str

1. 정수와 실수

- 정수는 -2, -1, 0, 1, 2, ...처럼 소수 부분이 없는 수
- 실수는 수학에서 정수·소수·무리수 등을 포함하는 수 체계
- Python의 float는 그중 일부를 유한한 정밀도로 근사해 저장하는 부동소수점 자료형

2. 사용

- 평균 성적처럼 소수 계산이 필요한 값을 구분해야 올바른 연산과 해석이 가능

"2022127046" -> str:숫자 모양 문자열

```
students = 8
average = 3.0
print(type(students).__name__)
print(type(average).__name__)
print(3 == 3.0)
print(7 / 2, 7 // 2)
```

int
float
True -> 자료형은 다르지만 값은 같음
3.5 3 -> / : 일반 나눗셈, // : 몫 연산자

1. 실수 계산의 주의점

- 부동소수점 오차는 유한한 이진 자릿수로 소수로 표현하면서 발생할 수 있음

2. 사용

- 계산 결과를 ==로 비교하거나 여러 번 반올림하면 예상과 다른 결과가 생길 수 있음
- 저장 오차와 표시 자릿수를 이해하고, 계산 중 불필요한 반올림을 피해야 함.

```
import math
value = 0.1 + 0.2
print(value)
print(value == 0.3)
print(f"{value:.2f}")
```

```
0.30000000000000004
False
0.30
```

```
print(round(value,2) == 0.3)

math.isclose(value, 0.3)
```

f"..." → f-string, 문자열 안에 변수 값을 넣는 방식

{value} → value 변수의 값을 넣음

: → 출력 형식을 지정하겠다는 뜻

.2f → 실수형(float)을 소수점 둘째 자리까지 표시

1. 문자열, Boolean, None

- 문자열(str) : 글자의 나열
- Boolean : True 또는 False
- None : 값이 없음을 표현하는 Python의 객체

2. 사용

- True, False는 조건식의 결과
- None, 0, 빈 문자열은 서로 다른 값을 의미

```
print(2 < 3)                    True
print(None == 0)                False
print(bool("False"))            True
print(bool(""))                 False
```

-> 학점이 없는 학생을 0으로 변환하면? 실제 0학점과 구분할 수 없어서 무조건 0으로 변환하면 안됨.

1. 자료형 변환

- 기존 값을 int·float 등 다른 종류의 값으로 바꾸는 것
- 표현이 해당 형식에 맞지 않으면 변환이 실패할 수 있음

2. 사용

- **읽은 숫자가 문자열로 되어 있다면 수식 계산이 안되서 숫자로 변환 필요**

```
raw_scores = pd.Series(["3.5", "미입력", "2.0"])
num_scores = pd.to_numeric(raw_scores)
```

-> ValueError: Unable to parse string "미입력" at position 1

```
num_scores = pd.to_numeric(raw_scores, errors="coerce")
```

-> [3.5, nan, 2.0]

errors="raise" → 기본값, 변환 실패 시 에러 발생

errors="coerce" → 변환 실패 값을 NaN으로 변환

errors="ignore" → 예전에는 원본 유지 용도로 사용

1. **자료구조 :** 여러 값을 저장하고 찾는 방식을 정한 구조
   - 리스트 : 순서 목록
   - 딕셔너리 : 키값과 값을 가지고 있는 원소 모음
   - 튜플 : 원소 참조를 바꿀 수 없는 순서 모음(하나의 의미 있는 묶음을 만들 때 많이 씀)
   - 집합 : 중복 제거 후 원소 모음

2. **사용**
   - **학생 목록 : 리스트 사용**
   - **학생의 속성 : 딕셔너리 사용,** `student = {“std_no": “1111", "gpa": 4.5}`
   - **학과목록 : 집합 사용(중복 제거를 위해)**

```
departments = ["A", "A", "B"]
student = {"id": "S01", "gpa": 3.8}
term = (2026, 1)
print(sorted(set(departments)))
print(student["id"])
print(term[0])
```

```
['A', 'B']
S01
2026
```

```
t = (1, [2, 3], 4)
t[1] = [10, 20]
```

```
TypeError: 'tuple' object does not support item assignment
```

1. 대입, 복사

- 대입 : 새 이름을 같은 객체에 연결
- 복사 : 별도의 객체를 만드는 것

2. 사용

- 복사를 사용하는 이유는 분석용 데이터를 수정했는데 원본 데이터까지 바뀌면 원본 데이터를 바탕으로 다른 수식을 하거나 전후 비교를 하기 위해 필요함

```
a = [1, 2]
alias = a
alias.append(3)
print(a)
copied = a.copy()
copied.append(4)
print(a, copied)
```

```
[1, 2, 3]
[1, 2, 3] [1, 2, 3, 4]
```

```
a = [[1, 2], [3, 4]]
b = a.copy()
print(a is b)
print(a[0] is b[0])
```

```
False
True
```

1. 리스트 컴프리헨션(List Comprehension)

- 반복하면서 계산하거나 조건에 맞는 원소를 골라 새 리스트를 만드는 표현식
- **특정 학점 이상의 학생, 특정 학과의 학생**

2. 사용

- 분석에 사용할 열 목록이나 조건에 맞는 값 목록을 간결하게 만들때 사용

```
scores = [3.8, 2.1, 3.2]
selected_scores = [x for x in scores if x >= 3.0]
print(selected_scores)
```

[3.8, 3.2]

```
students = [
{"name": "김민수", "department": "컴퓨터공학과"},
{"name": "이서연", "department": "경영학과"},
{"name": "박지훈", "department": "컴퓨터공학과"},
{"name": "최유진", "department": "국어국문학과"}]
selected_students = [
student    for student in students    if student["department"]
== "컴퓨터공학과"]
print(selected_students)
```

[{'name': '김민수', 'department': '컴퓨터공학과'},
{'name': '박지훈', 'department': '컴퓨터공학과'}]

1. Lambda, map

- 함수 이름을 별도로 정의하지 않고 하나의 표현식으로 함수 객체를 만드는 문법
- 여러 데이터의 각 요소에 같은 함수를 하나씩 적용하는 함수

2. 사용

- 정렬 기준이나 간단한 값 변환처럼 함수를 잠깐 전달해야 할 때 코드를 짧게 표현할 수 있음
- 코드값을 일관되게 변경할때 map 사용

```
import pandas as pd

df = pd.DataFrame({ "id": ["S01", "S02", "S03", "S04"],
"gpa": [3.8, 2.1, 3.2, 1.9],
"dropout": [0, 1, 0, 1],
"credits": [120, 60, 90, 45] })
records = df.to_dict("records")
print([r["id"] for r in sorted(records, key=lambda r: r["gpa"])])
labels = df["dropout"].map({ 0: "비탈락", 1: "탈락" })
print(labels.head(2).tolist())
```

['S04','S02', 'S03','S01']

['비탈락', '탈락']

lambda

```
def get_gpa(record):
        return record["gpa"]
sorted_records = sorted(records, key=get_gpa)
result = []
for r in sorted_records:
        result.append(r["id"])
print(result)
```

map

```
labels = []
for value in df["dropout"]:
        if value == 0:
                labels.append("비탈락")
        elif value == 1:
                labels.append("탈락")
```

1. 객체지향이란?

- 객체지향 프로그래밍은 관련 데이터인 상태와 그 데이터를 다루는 동작을 객체 단위로 묶어 프로그램을 구성하는 방식 **"객체 = 프로그램에서 표현한 하나의 대상"**

2. 사용

- 규칙과 데이터가 여러 곳에 흩어지는 문제를 줄임
- 비슷한 작업을 반복하고, 기능을 구조적으로 쉽게 관리할 수 있도록 사용함

- 객체 = 상태 + 동작
- 클래스 = 객체의 구조와 동작을 정의하는 틀
- 인스턴스 = 클래스로 만든 구체적인 객체
- class Student:

```
    def __init__(self, student_id, gpa):
        self.student_id = student_id
        self.gpa = gpa

    def has_gpa(self):
        return self.gpa is not None
s1 = Student("S01", 3.8)
s2 = Student("S02", None)
```

Student → 클래스
Student("S01", 3.8) → Student 클래스로 생성된 객체(인스턴스) s1 → 그 객체를 가리키는 변수명(이름)
Student("S02", None) → 또 다른 객체(인스턴스)
s2 → 그 객체를 가리키는 변수명(이름)

상태(데이터)- student_id = "S01"- gpa = 3.8
동작(메서드)- has_gpa()

개념	설명	데이터 분석 예
캡슐화	데이터와 기능을 하나로 묶음	DataFrame과 분석 함수를 Analyzer 클래스에 묶음
추상화	복잡한 내부 과정은 숨기고 필요한 기능만 제공	analyze()만 호출하면 전처리부터 결과 계산까지 수행
상속	기존 클래스의 기능을 물려받아 확장	공통 분석 클래스를 학생·예산·KPI 분석기가 상속
다형성	같은 호출이 객체에 따라 다르게 동작	모든 분석기를 analyze()로 호출하지만 분석 방법은 서로 다름

1. 캡슐화(Encapsulation)

: 데이터와 그 데이터를 처리하는 기능을 하나로 묶고, 외부에서 함부로 건드리지 않도록 사용 경계를 정하는 것

```
class StudentAnalyzer:
    def __init__(self, df):
        self._df = df
    def average_gpa(self):
        return self._df["gpa"].mean()
    def dropout_rate(self):
        return self._df["dropout"].mean()
    def update_gpa(self, student_id, new_gpa):
        if 0.0 <= new_gpa <= 4.5:
            self._df.loc[self._df["id"] == student_id, "gpa"] = new_gpa
        else:
            print("GPA는 0.0 ~ 4.5 사이여야 합니다.")
```

_ "이 속성은 클래스 내부에서 사용하는 용도이니 외부에서 직접 만지지 않는 것이 좋다"

- 값을 외부에서 직접 바꾸기 어렵게 하고, 정해진 메서드를 통해서만 변경하게 하는 것

```
analyzer = StudentAnalyzer(df)
print(analyzer.average_gpa())
print(analyzer.dropout_rate())
analyzer.update_gpa("1",4.8)
```

1. 추상화(Abstraction)

: 추상화는 **복잡한 내부 처리 과정은 숨기고 사용자가 필요한 기능만 보여주는 것**

```
class StudentAnalyzer:
    def __init__(self, df):
        self._df = df

    def dropout_rate(self):
        data = self._df["dropout"].dropna()
        data = pd.to_numeric(data, errors="coerce")
        data = data.dropna()

        return data.mean()
```

- 사용자는 내부 과정을 신경쓰지 않고 실행만 하면

(결측값 제거 되고, 오류값 제거 되고, 평균 계산되는 일련의 과정이 진행됨)

```
analyzer = StudentAnalyzer(df)
print(analyzer.dropout_rate())
```

1. 상속

: 공통적으로 필요한 기능을 부모 클래스에 두고 각각의 별도의 기능을 하는 클래스를 만들때 사용

```
class BaseAnalyzer:
    def __init__(self, df):
        self.df = df
    def row_count(self):
        return len(self.df)
    def missing_count(self):
        return self.df.isna().sum()


class DropoutAnalyzer(BaseAnalyzer):
    def dropout_rate(self):
        return self.df["dropout"].mean()
```

- DropoutAnalyzer에는
  row_count()를 직접 작성하지 않았는데도 사용할 수 있음

```
dropout_analyzer = DropoutAnalyzer(df)

print(dropout_analyzer.row_count())
print(dropout_analyzer.dropout_rate())
```

1. 다형성(Polymorphism)

```
class LogisticModel:
        def predict(self, X):
                return "로지스틱 회귀 예측"
class RandomForestModel:
        def predict(self, X):
                return "랜덤포레스트 예측"
class XGBoostModel:
        def predict(self, X):
                return "XGBoost 예측"
```

```
models = [
    LogisticModel(),
    RandomForestModel(),
    XGBoostModel()]

for model in models:
    print(model.predict("학생 데이터"))
```

이제 사용하는 쪽에서는 모델 종류를 일일이 따질 필요 없이 모두 똑같이 predict()만 호출하면 됩니다.

1. 데이터 점검

- head(): 데이터프레임의 앞부분 행을 확인하는 메서드. 기본적으로 처음 5개 행을 보여줌
- info(): 데이터프레임의 행/열 개수, 컬럼명, 데이터 타입, 결측치 여부, 메모리 사용량 등을 요약해서 보여주는 메서드
- shape: 데이터프레임의 행 개수와 열 개수를 (행, 열) 형태의 튜플로 반환하는 속성

2. 사용

- head(): 값이 예상대로 들어왔는가?
- info(): 자료형과 결측 상태는 어떤가?
- shape: 학생 수와 변수 수가 맞는가?

- df = pd.DataFrame({ "id": ["S01", "S02", "S03"], "gpa": [3.8, np.nan, 3.2] })

- print(df.head())
- df.info()
- print(df.shape)

```
 id  gpa
0  S01  3.8
1  S02  2.1
2   S03  3.2

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
 0   id      3 non-null      object
 1   gpa     2 non-null      float64
dtypes: float64(1), object(1)

(3, 2)
```

1. apply·lambda와 열 연산

- apply는 함수 적용 인터페이스이며, DataFrame에서 axis=1을 지정하면 행 단위 함수를 적용
- 열 연산은 열 전체를 대상으로 계산식을 표현하는 방식

2. 사용

- 같은 결과더라도 실행 비용이 다름
- 간단한 산술을 열 연산으로 보통 사용

- df["credits_result"] = df["credits"] / 18
- df["credits_result"] = df["credits"].apply(lambda x: x / 18)
- df["credits_result"] = df["credits"].apply(lambda x: x / 18 if x >= 60 else x)

- 첫번째처럼 쓰면 credits 열 전체를 하나의 벡터처럼 보고 연산한다고 해서 벡터화 연산(vectorized operation)
- 벡터화 연산은 여러 값을 하나의 벡터(배열)로 보고, 개별 반복문 없이 전체 데이터에 연산을 적용하는 방식
- 두번째는 겉으로 보기에는 열 전체에 적용하지만, 개념적으로는 각 원소를 하나씩 함수에 넣는 방식

1. 그룹별 집계

- groupby() : 집단을 나눔
- agg() : 여러 지표를 함께 계산

2. 사용

- 집계를 못 하는 것이 아니라, 여러 집계 결과를 한 번에 정리해서 계산하기 편하게 해주는 메서드

```
summary = df.groupby("department").agg(
    students=("student_id", "size"),
    gpa_n=("gpa", "count"),
    mean_gpa=("gpa", "mean"),
    dropout_rate=("dropout", "mean")
)
```

```
students = df.groupby("department")["student_id"].size()
gpa_n =df.groupby("department")["gpa"].count()
mean_gpa = df.groupby("department")["gpa"].mean()
dropout_rate = df.groupby("department")["dropout"].mean()

summary = pd.DataFrame({
"students": students,
"gpa_n": gpa_n,
"mean_gpa": mean_gpa,
"dropout_rate": dropout_rate})
```

department	students	gpa_n	mean_gpa	dropout_rate
컴퓨터공학	30	28	3.42	0.10
경영학	40	39	3.25	0.15

함수	의미
sum	합계
mean	평균
median	중앙값
min	최솟값
max	최댓값
count	결측치 제외 개수
size	전체 행 개수
std	표준편차
var	분산
nunique	고유값 개수
first	첫 번째 값
last	마지막 값

1. 표 결합

- merge() : 공통 열을 기준으로 표를 결합

2. 사용

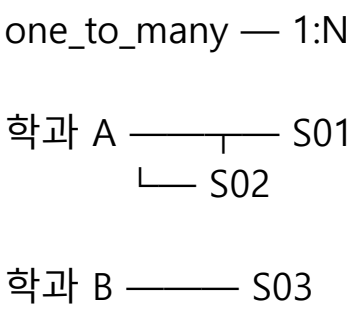
- 서로 다른 데이터프레임을 공통 기준 열(key)을 이용해 하나로 결합할 때 사용

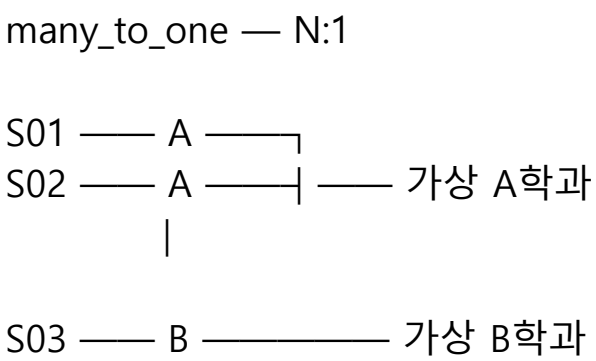
```
df = pd.DataFrame({
    "student_id": ["S01", "S02", "S03", "S04", "S05"],
    "department": ["A", "A", "B", "B", "C"],
    "gpa": [3.8, 3.2, 2.9, 3.5, 2.7]
})
dept = pd.DataFrame({
    "department": ["A", "B"],
    "dept_name": ["가상 A학과", "가상 B학과"]
})
merged = df.merge(
    dept,
    on="department",
    how="left",
    validate="many_to_one"
)
print(merged)
print(len(df), len(merged))
print(merged["dept_name"].isna().sum())
```

```
  student_id department  gpa    dept_name
0        S01          A  3.8   가상 A학과
1        S02          A  3.2   가상 A학과
2        S03          B  2.9   가상 B학과
3        S04          B  3.5   가상 B학과
4        S05          C  2.7          NaN

5 5
1
```

validate	관계	왼쪽 키	오른쪽 키	예
"one_to_one"	1:1	유일	유일	학생번호 ↔ 학생 정보
"one_to_many"	1:N	유일	중복 가능	학과 ↔ 학생
"many_to_one"	N:1	중복 가능	유일	학생 ↔ 학과
"many_to_many"	N:M	중복 가능	중복 가능	학생 ↔ 수강과목





10분간

파이썬 실습

탐색적 데이터분석 (EDA)

EDA란 무엇인가?

- 탐색적 데이터 분석은 형식적 모형을 먼저 고정하기보다 다양한 관점에서 데이터를 바라보며 구조, 예외, 가정 위반, 새로운 질문을 발견하는 접근

- 계산과 그래프는 목적이 아니라 데이터에 대한 통찰을 얻기 위한 수단
- 평균과 표준편차가 같아도 분포 모양과 변수 관계는 다를 수 있다.
- 그래프는 비선형성, 혼합분포, 군집, 꼬리, 이상치, 시간 순서 등 요약값이 숨길 수 있는 구조를 드러낸다.

■ 목표

✓ 데이터에 대한 통찰을 최대화한다.
✓ 잠재적 구조를 발견한다.
✓ 중요한 변수를 추출한다.
✓ 이상값과 특이한 관측을 탐지한다.
✓ 기본 가정을 검정한다.
✓ 간결한 모형을 개발하고 최적 요인 설정을 탐색한다.

고전적분석과 EDA의 차이점

고전적 분석 : 문제 → 데이터 → 모형 → 분석 → 결론
EDA : 문제 → 데이터 → 분석 → 모형 → 결론

관점	고전적 분석	EDA
모형	결정론적·확률적 모형을 데이터에 부과	데이터가 적절한 모형을 제안하도록 탐색
초점	모수 추정과 모형 기반 예측	데이터 구조, 이상치, 후보 모형
기법	ANOVA, t, 카이제곱, F 검정 등 정량 기법	산점도, 상자그림, 히스토그램, 확률도 등 그래픽 기법
엄밀성	형식적이고 확률론적으로 엄밀한 결론	해석 의존성이 있으나 모형 발견에 매우 시사적
데이터 처리	데이터를 소수 추정값으로 축약하여 일부 구조가 걸러질 수 있음	가능한 모든 데이터를 보여 왜도·꼬리·자기상관도 살핌
가정	검정력과 결론이 가정의 타당성에 의존	자료를 있는 그대로 보여주는 기법은 상대적으로 가정이 적음

EDA의 단계

단계	내용	핵심
1. 분석 문제 정의	분석 목적과 해결하려는 문제를 명확히 함	무엇을 알고 싶은가?
2. 분석 범위와 데이터 정의	대상, 분석 단위, 기간, 결과변수, 사용 가능한 변수 등 확인	어떤 데이터를 대상으로 볼 것인가?
3. 주요 분석 질문 선정 및 우선순위 설정	NIST의 13가지 질문 중 관련 질문을 선택	무엇부터 확인할 것인가?
4. EDA 수행	질문에 맞는 그래프와 통계량 사용	어떻게 확인할 것인가?

EDA 과정

1. 문제 정의
"중도탈락 학생은 어떤 특성을 보이며, 어떤 요인이 중도탈락과 관련되는가?"
↓
2. 분석 범위 명확화
2015년 이후 입학생 / 학생 1명 단위 / 2025.4.1 기준 / GPA·이수학점·휴학 등의 정보 사용
↓
3. NIST의 13개 질문에서 관련 질문 선택
•전형적인 값은 무엇인가?
•특정 요인이 영향을 미치는가?
•가장 중요한 요인은 무엇인가?
•여러 집단의 결과가 동등한가?
•다변량 데이터에서 구조를 발견할 수 있는가?
•이상치가 존재하는가?
↓
4. 구체적인 EDA 질문으로 변환
•중도탈락자의 GPA 중앙값은 얼마인가?
•중도탈락자와 재학생의 GPA 분포가 다른가?
•학사경고 경험과 중도탈락은 관련되는가?
•어떤 변수가 중도탈락과 가장 강하게 관련되는가?
•학생 특성에 따라 서로 다른 군집이 나타나는가?
•극단적인 이수학점이나 GPA 값이 존재하는가?
↓
5. 분석 방법 선택
히스토그램, Box plot, Scatter plot, 그룹별 요약통계, 상관관계 등

분석 범위 명확화(분석에 사용할 데이터의 범위와 기준)

구분	확인할 내용	중도탈락 분석 예시
분석 대상	누구를 분석할 것인가	2015년 이후 입학 학부생
분석 단위	한 행이 무엇을 의미하는가	학생 1명
관찰 기간	어느 기간의 데이터를 사용할 것인가	입학 후 ~ 2025.04.01
기준 시점	어느 시점을 기준으로 정보를 확정하는가	2025.04.01 스냅샷
포함·제외 기준	어떤 데이터를 포함하거나 제외할 것인가	졸업생 제외, 재학생·중도탈락자 포함
결과변수 정의	무엇을 결과로 판단할 것인가	다음 학기 이전 중도탈락 여부
설명변수 범위	어떤 정보를 분석에 사용할 것인가	GPA, 이수학점, 휴학, 상담횟수 등
정보 사용 시점	결과 이후 정보를 사용하지 않는가	중도탈락 이후 발생 변수 제외
비교 기준	무엇과 무엇을 비교할 것인가	중도탈락 학생 vs 재학생
집단 구분	세부 집단 분석이 필요한가	학년·단과대학·입학전형별
데이터 품질 기준	어떤 데이터까지 신뢰할 것인가	결측·중복·비정상 값 확인
해석 범위	결과를 어디까지 일반화할 것인가	해당 대학의 2015년 이후 입학생

데이터 구조 및 품질 진단

1. 결측값
  - 결측은 단순한 빈칸이 아니라 데이터 생성 과정의 결과다. 미응답, 해당 없음, 측정 실패, 시스템 연계 실패를 구분해야 한다. 결측률만으로 삭제 또는 대체를 결정하지 말고 목표변수 및 주요 집단과 결측 여부의 관계를 확인 필요

  잘못된 대응 : 일괄 평균 대체
   -> 원인과 패턴 확인 후 변수별 처리

2. 중복값
  - 데이터의 완전 중복? 키값이 중복?

  잘못된 대응 : 모두 삭제
   -> 분석 단위와 키를 먼저 확인 후 처리

3. 허용 범위
  - 학점 만점이 4.5인 경우인데 4.5 초과를 한 데이터가 입력

  잘못된 대응 : 자동 클리핑
   -> 원천 오류 여부와 업무 규칙 확인

4. 논리적 일관성
 - 취득학점이 음수인 경우

  잘못된 대응 : 무조건 기타 통합
   -> 의미와 예측, 공정성 영향 확인

분석 전에 필요한 질문(주로 사용되는 질문들)

번호	주요 분석 질문	대학 중도탈락 데이터 적용 예시	확인 이유 및 활용
1	전형적인 값은 무엇인가?	전체 학생의 평균·중앙 GPA와 취득학점은 얼마인가?	학생 집단의 일반적인 학업 수준을 파악하고 비교 기준을 설정한다.
2	전형적인 값의 불확실성은 어느 정도인가?	평균 GPA와 중도탈락률의 95% 신뢰구간은 얼마인가?	표본에서 계산한 값이 얼마나 안정적인지 판단하고 모집단의 값을 추정한다.
3	어떤 확률분포가 자료에 잘 맞는가?	상담 횟수는 정규·포아송·음이항분포 중 어디에 가까운가?	변수에 적합한 통계기법과 변환 및 이상치 판단 기준을 선택한다.
4	특정 백분위수는 얼마인가?	동일 학년 학생 중 취득학점 하위 10%의 기준은 몇 학점인가?	위험군 기준이나 상·하위 집단의 분류 기준을 객관적으로 설정한다.
5	공정 또는 제도의 변경이 효과가 있는가?	학습지원 프로그램 시행 전후에 중도탈락률이 감소했는가?	제도나 프로그램의 성과를 평가하고 지속·확대 여부를 결정한다.
6	특정 요인이 반응에 영향을 주는가?	학사경고 경험에 따라 중도탈락률에 차이가 있는가?	중도탈락과 관련된 개별 요인을 발견하고 지원 대상을 구체화한다.
7	가장 중요한 요인은 무엇인가?	GPA, 취득학점, 휴학 횟수 중 중도탈락과 가장 밀접한 변수는 무엇인가?	위험 요인의 우선순위를 정하고 한정된 지원 자원을 효율적으로 배분한다.
8	서로 다른 기관이나 측정 결과가 동등한가?	단과대학별 GPA 산정 기준과 상담 기록 기준이 일관적인가?	집단 간 비교가 가능한지 확인하고 측정 및 입력 기준의 차이를 발견한다.
9	반응변수와 요인변수를 연결하는 적절한 함수는 무엇인가?	취득학점과 GPA로 중도탈락 확률을 어떤 함수로 설명할 수 있는가?	선형·비선형 관계를 파악하여 적합한 분석모형과 예측모형을 선택한다.
10	요인의 최적 설정은 무엇인가?	중도탈락 위험을 낮추기 위한 적정 상담 횟수는 얼마인가?	효과와 비용을 함께 고려하여 프로그램 운영 기준을 설정한다.
11	시간 의존 자료에서 신호와 잡음을 구분할 수 있는가?	로그인 횟수의 지속적인 감소가 위험 신호인지 일시적 변동인지 구분할 수 있는가?	추세·계절성·일시적 변동을 구분하여 조기경보의 오탐을 줄인다.
12	다변량 자료에서 구조를 추출할 수 있는가?	학업부진형·대학생활 미참여형 등 학생 유형을 발견할 수 있는가?	여러 변수의 조합으로 잠재 집단을 찾아 유형별 맞춤 지원을 설계한다.
13	이상치가 존재하는가?	GPA 45.0, 음수 취득학점 등 비정상적인 기록이 있는가?	입력 오류와 중요한 희귀 사례를 구분하여 분석 결과의 왜곡을 방지한다.

구체적인 EDA 질문으로 변환 이후 흐름

변수의 관계를 파악하여 분포를 이해하고, 관계를 찾은 뒤, 불확실성과 분석 가정을 확인한다

단계	핵심 질문	결과
단변량 EDA	변수 하나는 어떤 모습인가?	대표값·산포·분포 해석
이변량·다변량 EDA	어떤 차이와 관계가 있는가?	비교표·관계 그래프·조건부 해석
확률분포와 검정	우연한 변동으로도 설명되는가?	효과 크기·신뢰구간·검정 결과
분포 변환	어떤 척도에서 분석이 적절한가?	변환 전후 비교와 선택 근거

필요하면 품질 점검이나 단변량 분석으로 돌아간다

구체적인 EDA 질문으로 변환 이후 흐름

단변량 EDA
- 각 변수의 분포와 특성을 이해한다.
- 평균, 중앙값, 표준편차, IQR, 이상치 등을 확인한다.
- 예: GPA는 어떤 분포를 보이는가?

이변량·다변량 EDA
- 변수 간 관계와 집단 간 차이를 탐색한다.
- 상관관계, 그룹별 분포 차이, 변수 간 패턴을 확인한다.
- 예: 탈락/비탈락 집단의 GPA는 다른가?

확률분포와 불확실성 이해
- 관측값과 표본 통계량이 얼마나 변동할 수 있는지 이해한다.
- 확률분포와 표집분포를 통해 불확실성을 파악한다.
- 예: 표본평균은 표본을 다시 뽑으면 얼마나 달라질 수 있는가?
  -> 예를 들어 중도탈락 학생과 비탈락 학생의 평균 GPA가 다르게 나왔다고 해도 0.4점 차이로 두 집단을 다르다고 결론을 내리면 안됩니다.
  표본을 다시 뽑으면 평균이 조금씩 달라질 수 있기 때문에.

통계적 검정
- EDA에서 발견한 차이와 관계가 우연인지 확인한다.
- 데이터 특성과 가정에 맞는 검정 방법을 선택한다.
- 예: t-test, 카이제곱 검정

분포 변환
- 왜도, 이상치, 분산 불균형 등이 큰 경우 분포를 조정한다.
- 분석 가정을 만족시키거나 관계를 더 명확하게 만들기 위해 적용한다.
- 예: log 변환

공통 실습 데이터와 해석 범위

가상 학생 8명: 한 행은 학생 1명, GPA 결측은 1명

학생	학과	GPA	이수학점	탈락(1)
S01	A	3.8	18	0
S02	A	2.1	9	1
S03	A	3.2	15	0
S04	A	결측	12	0
S05	B	1.8	6	1
S06	B	3.6	18	0
S07	B	2.5	12	0
S08	B	3.0	15	0

GPA 요약의 분모는 7명, 탈락률의 분모는 8명이다

단변량 EDA의 네 가지 관점

무엇이 보통이고, 얼마나 다르며, 어디가 특이한가?

관점	살펴볼 내용	먼저 사용할 방법
위치	값이 주로 모이는 곳	평균·중앙값
산포	값이 퍼져 있는 정도	표준편차·IQR
모양	대칭성·꼬리·봉우리 수	히스토그램
특이값	다른 값에서 멀리 떨어진 관측	상자그림과 원자료 확인

수치형은 분포, 범주형은 빈도와 비율부터 확인한다

-> 변수의 특징을 이해한다.

Q1·중앙값·Q3와 IQR의 계산

별도 가상 GPA 9개를 정렬한 뒤, 자료를 네 부분으로 나누는 경계값을 확인한다

순서	1	2	3	4	5	6	7	8	9
GPA	1.0	1.5	2.0	2.5	3.0	3.2	3.5	3.8	4.0
경계			Q1		중앙값		Q3		

통계량	값	의미
Q1 (25% 분위수)	2.0	작은 쪽 25%의 경계
중앙값 (50% 분위수)	3.0	정렬한 자료의 가운데
Q3 (75% 분위수)	3.5	작은 쪽 75%의 경계
IQR = Q3 − Q1	1.5점	중앙 50% 구간 2.0~3.5의 폭

중앙값은 전형적인 수준, IQR은 가운데 자료의 퍼짐을 요약한다. 소표본 분위수는 계산법을 명시한다.
"중앙값 하나만 보면 학생들의 가운데 수준만 알 수 있습니다. IQR을 함께 보면 가운데 학생들이 비슷한 수준에 모여 있는지, 넓게 퍼져 있는지도 알 수 있습니다. 그래서 집단을 요약할 때 두 값을 함께 봅니다."

- 활용 목적은 집단의 수준과 퍼짐을 비교하는 데 있습니다. 예를 들어 두 학과의 중앙값이 모두 3.0이어도 다음처럼 다를 수 있습니다.

극단값이 있을 때 평균과 중앙값의 역할

전형적인 수준은 중앙값·IQR로 보완하고, 평균은 목적에 맞게 함께 보고한다

별도 숫자 예제	평균	중앙값	IQR
10, 10, 10, 10, 10	10	10	0
10, 10, 10, 10, 100	28	10	0

왜 중앙값·IQR인가?

정렬의 가운데와 중앙 50%를 보므로 소수 극단값의 영향을 덜 받는다.

평균을 버리는 것은 아니다

전체 총량·평균 부담이 질문이면 평균도 중요하다. 100의 원인은 따로 확인한다.

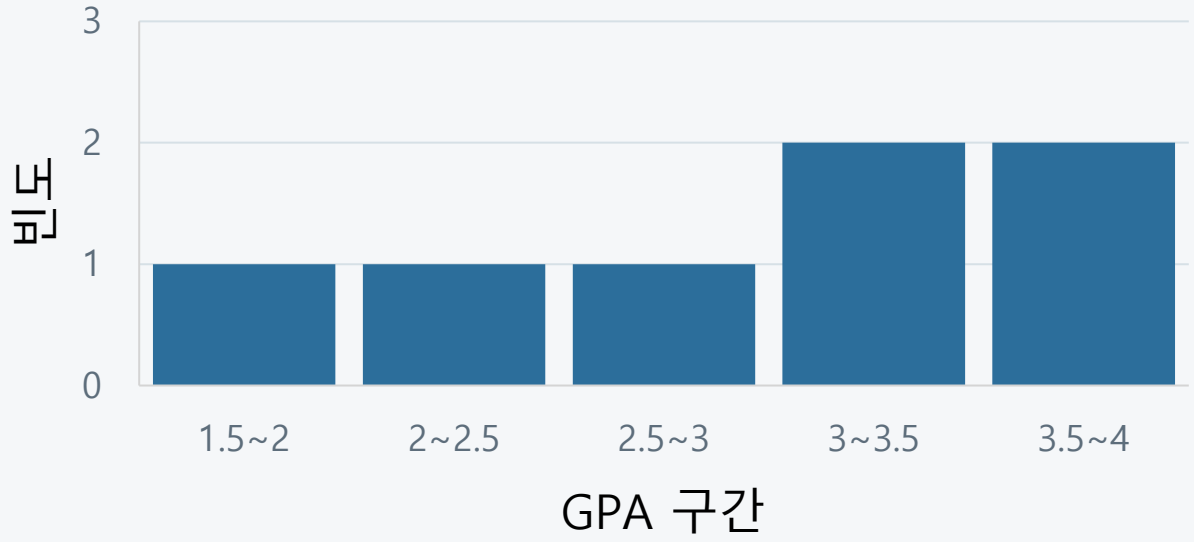
공통 GPA에 적용

평균 2.86, 중앙값 3.00, IQR 1.10. 대표값만으로 분포 모양을 단정하지 않는다.
- 예를 들어 평균이 중앙값보다 작다고 해서 바로 "왼쪽으로 긴 꼬리를 가진 분포다"라고 단정하면 안 됩니다. 실제 값이나 히스토그램을 함께 확인해야 합니다.

단변량 EDA 실습 결과

코드 출력과 그래프를 해석 문장으로 연결한다

항목	출력
count / 결측	7 / 1
mean / std	2.86 / 0.75
min / 25%	1.80 / 2.30
50% / 75% / max	3.00 / 3.40 / 3.80



해석: 유효 7명에서 평균은 2.86점, 중앙값은 3.00점이다.
인접한 3.0~4.0 구간에 4명이 있다. GPA 3.0 이하는 4/7=57.1%다.
다음 질문: 탈락 여부별로 GPA의 분포와 유효 건수가 어떻게 다른가?

단변량 실습 코드와 숫자 출력

42쪽 df의 GPA 결측을 제외하고 요약통계와 기준 이하 비율을 계산한다

```
x = df["gpa"].dropna()
print(x.describe().round(2))
iqr = x.quantile(.75) - x.quantile(.25)
print(f"IQR: {iqr:.2f}")
print(f"3.0 이하: {(x <= 3).mean():.1%}")

sns.histplot(x=x,
    bins=[1.5, 2, 2.5, 3, 3.5, 4])
plt.show()
sns.ecdfplot(x=x)
plt.show()
```

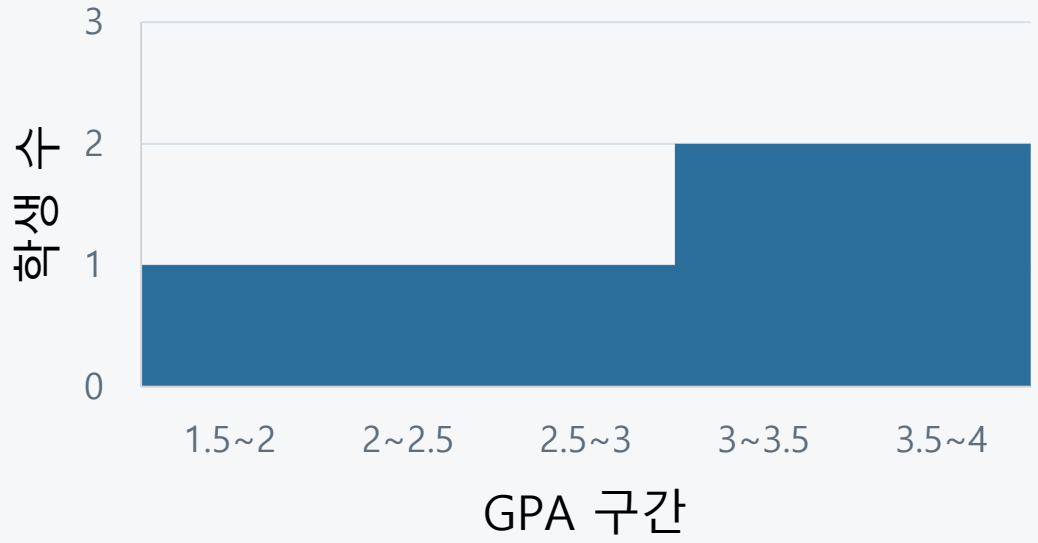
숫자 출력	값
count / mean / std	7 / 2.86 / 0.75
min / 25%	1.80 / 2.30
50% / 75% / max	3.00 / 3.40 / 3.80
IQR	1.10
3.0 이하	57.1%

숫자 출력에 이어 히스토그램과 ECDF를 각각 출력한다. 두 그래프는 다음 장에서 함께 읽는다

단변량 실습의 두 그래프와 해석

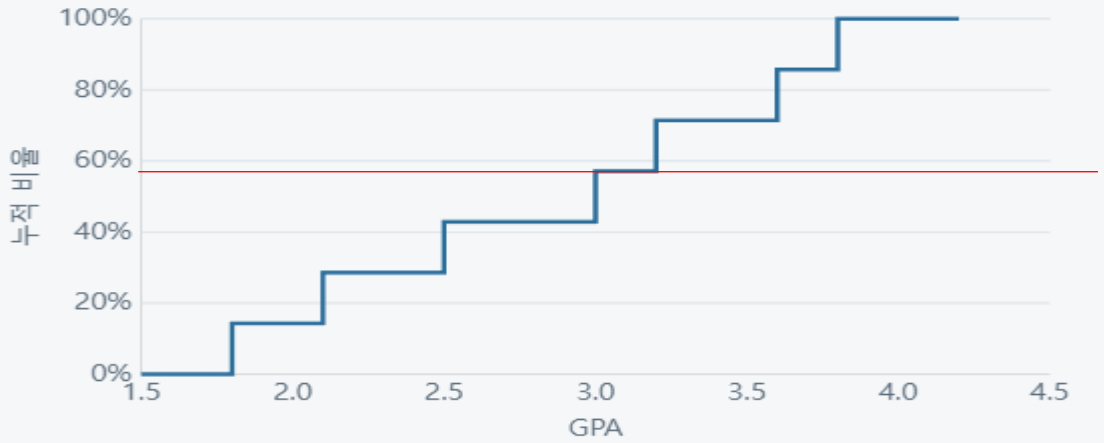
같은 유효 GPA 7명: 히스토그램은 구간별 건수, ECDF는 기준 이하 누적비율

히스토그램: 3.0~4.0 구간에 4명



GPA 3.0은 [3.0, 3.5)에 포함한다.
인접한 높은 막대만으로 두 봉우리를 단정하지 않는다.

ECDF: 3.0 이하 4/7 = 57.1%



각 학생을 지날 때 1/7씩 올라간다.
GPA 3.0에서 곡선의 높이는 57.1%다.

표본 크기가 다르거나 임계값을 비교할 때 유용하다

첫째, 기준값 이하에 얼마나 많은 데이터가 몰려 있는지 바로 알 수 있습니다.
평균만 보면 전체 수준은 알 수 있지만, "3.0 이하 학생이 얼마나 되는가?"는 알기 어렵습니다. ECDF는 이런 기준값 기반 해석에 강합니다.
둘째, 위험 기준이나 정책 기준과 연결하기 좋습니다. 예를 들어 GPA 3.0 이하를 학업위험군으로 본다면
전체의 57.1%가 위험 기준 이하에 있다

구체적인 EDA 질문으로 변환 이후 흐름

예를 들어 학년별로 나누었더니 1학년은 2.0 부근, 4학년은 3.5 부근에 모여 있다고 가정하겠습니다.
그러면 전체의 두 봉우리는 학년별로 다른 분포가 합쳐져 나타난 것으로 설명할 수 있습니다.

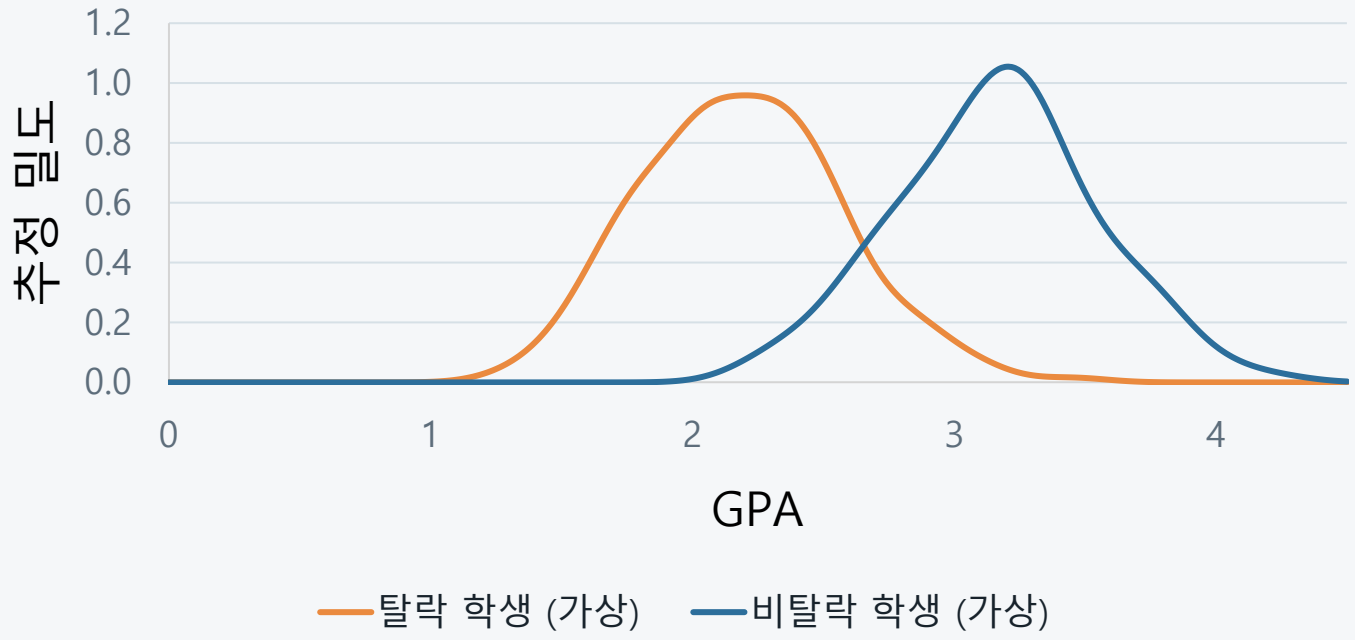
이때는 전체 평균만 제시하지 말고, 학년별 학생 수·중앙값·IQR·히스토그램을 함께 제시하는 것이 좋습니다.
중도탈락과의 관계를 분석한다면 질문도 구체화합니다.
"전체에서 GPA가 낮은 학생의 탈락률이 높은가?"
→ "같은 학년 안에서도 GPA가 낮은 학생의 탈락률이 높은가?
"학년 차이와 GPA의 관계를 구분해 보기 위한 것입니다.

예측모형에서는 GPA와 학년을 함께 사용하는 방법 등을 검증할 수 있지만, 두 봉우리가 있다는 이유만으로 모형을 두 개로 나누지는 않습니다.

KDE의 활용: 두 집단의 GPA 분포 비교

KDE(Kernel Density Estimation)
는 히스토그램처럼 분포를 보는 방법이며, 두 집단의 분포 모양을 부드러운 곡선으로 비교할 때 사용한다

탈락 학생과 비탈락 학생은 어느 GPA 구간에 모이는가? 설명용 가상 자료 각 200명



왜 곡선으로 보는가?

막대의 겹침을 줄여 두 집단의
중심·퍼짐·겹치는 구간을 비교한다.

이 예제에서 읽는 것

탈락 집단은 낮은 GPA에 모인다.
겹침이 있어 GPA만으로 구분은 어렵다.

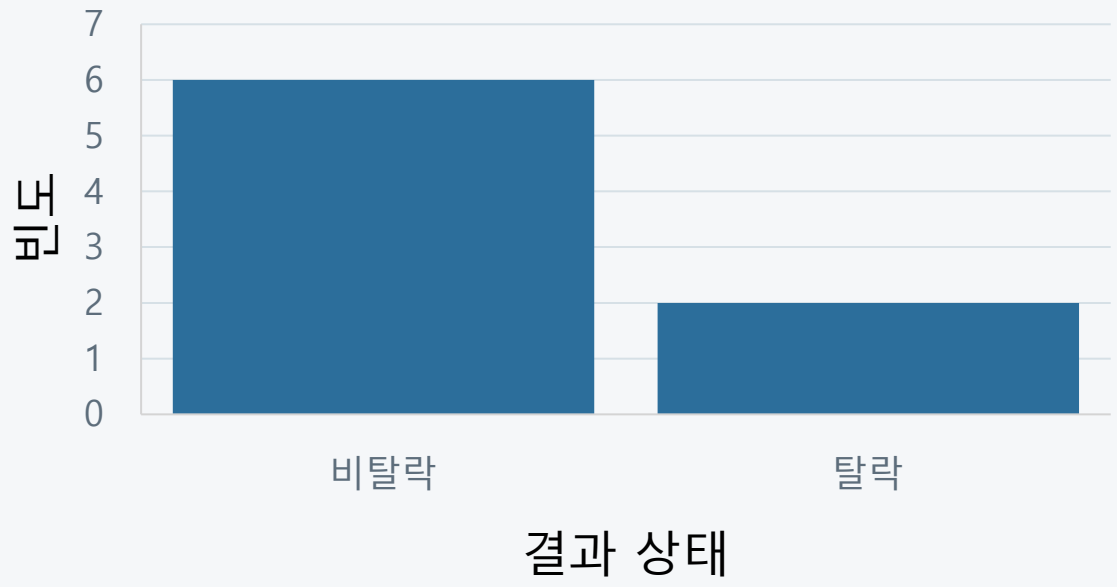
높이의 의미

0.4는 학생 40%라는 뜻이 아니다.
구간 아래 면적이 추정 비율이다.

KDE는 선택 도구다. 평활화와 GPA 경계의 영향을 확인하고, 표본 누적비율은 ECDF로 읽는다

범주형 변수에서 빈도와 비율을 보는 이유

학생 구성과 클래스 불균형을 파악하고, 비율의 분모를 분명히 한다



이 표본의 구성

비탈락 6/8=75%, 탈락 2/8=25%

왜 필요한가?

집단 크기를 알고 비교와 평가 지표를 계획한다.

현황과 일반화 구분

25%는 이 가상 표본의 사실이다.
대학 전체 비율 추정에는 대표성·기간 확인이 필요하다.

전부 비탈락으로 예측하면 이 표본 정확도는 75%, 탈락 재현율은 0%다

비탈락 학생이 더 많은 자료에서는 정확도만 보고 예측모형이 좋다고 판단하면 안 된다

이변량 EDA의 도구 선택

두 변수의 유형을 확인하면 첫 그래프를 고르기 쉽다

변수 조합	첫 도구	이 도구로 읽는 것
수치형 × 수치형	산점도 + 상관계수	관계의 방향·형태·예외
범주형 × 수치형	상자그림 + 집단 요약	집단의 중심·퍼짐·겹침
범주형 × 범주형	교차표 + 비율 막대	분모를 맞춘 구성비 차이
시간 × 수치형	시계열 선그래프	추세·급변·주기

상관계수나 p값을 계산하기 전에 그래프와 표본 수를 확인한다

상관계수 계산의 원리

각 값이 평균보다 높은지 낮은지 비교하고, 함께 움직이는 정도를 표준화한다

가상 x	가상 y	x − 평균 x	y − 평균 y	편차의 곱
1	2	−1	−2	2
2	4	0	0	0
3	6	1	2	2

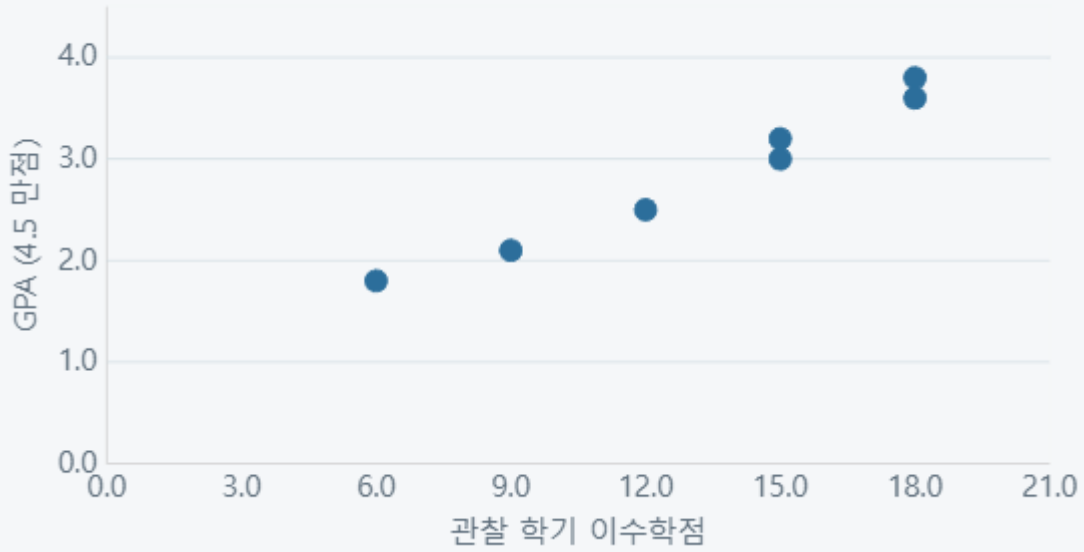
Pearson r = Σ[(x − 평균 x)(y − 평균 y)] / √{Σ(x − 평균 x)² × Σ(y − 평균 y)²}

위 예제: r = 4 / √(2 × 8) = 1

같은 방향이면 양수, 반대 방향이면 음수. 절댓값이 1에 가까울수록 직선에 가깝다

산점도와 상관계수

점 하나는 학생 1명이며, 좌표는 두 변수의 값이다



관찰

학점이 큰 학생의 GPA도 높다

수치 요약

Pearson r ≈ 0.986, 유효 n=7

추가 확인

극단값·집단 혼합·측정 시점

같이 움직인다는 관찰과 원인·결과 주장은 구분한다

이변량 EDA

산점도와 상관계수

"이수학점이 높은 학생의 GPA도 높게 나타났다"는 관찰만으로,

"이수학점을 늘리면 GPA가 높아진다"고 말할 수는 없다는 뜻입니다.

그래프가 나타나는 이유가 여러 가지일 수 있기 때문입니다.

- **다른 요인이 둘 다 높였을 수 있습니다.** 학습에 적극적인 학생이 학점도 많이 이수하고 GPA도 높게 받을 수 있습니다
- **영향의 방향이 반대일 수 있습니다.** 이전 GPA가 높은 학생이 더 많은 학점을 수강하고 이수했을 수 있습니다.
- **학년 등 집단 차이가 섞였을 수 있습니다.** 학년에 따라 이수학점과 GPA가 함께 달라질 수 있습니다.

상관행렬 히트맵

searborn.heatmap

	GPA	학점	경고	상담	대출	참여
GPA	1.00	0.94	-0.89	0.21	0.27	0.73
학점	0.94	1.00	-0.87	0.23	0.27	0.72
경고	-0.89	-0.87	1.00	-0.22	-0.28	-0.68
상담	0.21	0.23	-0.22	1.00	0.88	0.71
대출	0.27	0.27	-0.28	0.88	1.00	0.74
참여	0.73	0.72	-0.68	0.71	0.74	1.00

파랑: 양의 상관
주황: 음의 상관
연한 색: 0에 가까움

대각선은 1
위·아래는 대칭

GPA–학점처럼
강한 쌍은 산점도로
중복·극단값을 확인

상관 순위는 예측 중요도나 인과 영향력 순위가 아니다

공통 GPA와 이수학점의 상관 계산

두 변수 모두 있는 같은 학생의 쌍만 사용한다

```
xy = df[["credits", "gpa"]].dropna()
r = xy["credits"].corr(xy["gpa"], method="pearson")
print(len(xy), round(r, 3))
# 출력: 7  0.986
```

왜 두 열을 함께 dropna 하는가?

학생의 GPA와 다른 학생의 학점을 잘못 짝짓지 않기 위해서다.

계수와 점의 의미

0.986은 오른쪽 위로 거의 직선처럼 모이는 정도이다. 원인·결과의 증거는 아니다.

한 변수의 값이 모두 같으면 상관계수는 정의되지 않는다

집단별 분포 비교

평균 차이와 함께 분포의 겹침·산포·유효 n을 본다



비탈락

GPA 유효 5명, 평균 3.22

-> 학생마다 GPA 차이가 있다

탈락

GPA 유효 2명, 평균 1.95

-> 학생마다 GPA가 비슷하다

관찰의 한계

작은 표본의 차이를 일반화 금지

특히 탈락 집단은 한 학생의 값이 바뀌거나 한 명이 추가되어도 평균과 퍼짐이 크게 달라질 수 있습니다. 그러므로 **현재 관찰한 차이가 더 많은 학생에서도 유지되는지 추가 확인해야 합니다.** 결측 학생이 빠진 것이 비교에 영향을 주는지도 살펴봅니다.

교차표의 행 비율과 열 비율

같은 표라도 분모를 어디에 두는지에 따라 질문이 달라진다

학과	비탈락	탈락	학과 인원	학과 내 탈락률
A	3	1	4	1/4 = 25%
B	3	1	4	1/4 = 25%
합계	6	2	8	2/8 = 25%

행 비율: 학과 학생 중 탈락한 비율

pd.crosstab(df.department, df.dropout, normalize="index")

열 비율: 탈락자 중 해당 학과가 차지하는 비율

A학과 탈락 1명 / 전체 탈락 2명 = 50%

다변량 EDA가 필요한 이유

집단 구성이 다르면 전체 비교의 방향이 달라질 수 있다

사전 위험층	상담군 탈락 / 전체	비상담군 탈락 / 전체
저위험층	1/20 = 5%	8/100 = 8%
고위험층	30/100 = 30%	8/20 = 40%
전체	31/120 = 25.8%	16/120 = 13.3%

해석

상담군에는 고위험 학생이 더 많다(전체 25.8%). 같은 위험층 안의 비교도 함께 본다.

전체 비율 차이만으로 “상담이 탈락을 높인다”고 해석할 수 없다

산포와 유효 n

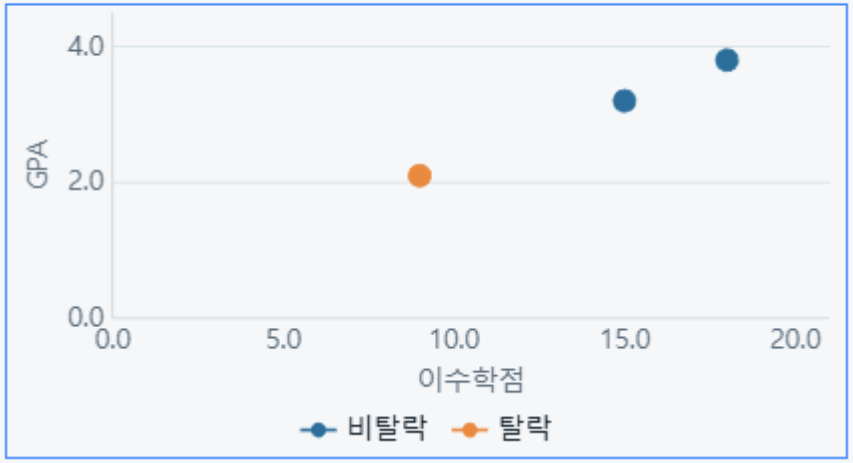
이 상태에서 전체 탈락률만 비교하면:
•상담군: **25.8%**
•비상담군: **13.3%**
상담군의 탈락률이 높아서 "상담이 오히려 안 좋은가?"라고 생각할 수 있습니다.

두 위험층 모두 상담군의 탈락률이 더 낮습니다. 그런데 상담군에 원래 탈락 위험이 높은 학생이 많이 포함되어 있어, 전체를 합치면 상담군의 탈락률이 더 높게 나타나는 것입니다.
-> "상담 여부와 탈락 여부만 비교하면 집단 구성의 차이를 놓칠 수 있습니다. 사전 위험 수준까지 함께 고려해, 비슷한 조건의 학생끼리 비교해야 합니다."

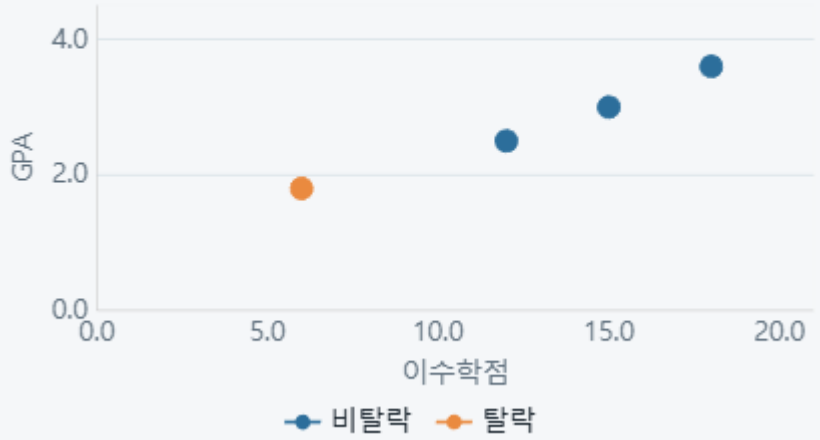
학과별 산점도로 조건을 나누어 비교

같은 축에서 학과를 나누고, 점의 색으로 탈락 여부를 구분한다

학과 A 유효 n=3



학과 B 유효 n=4



이 예제에서는 학과를 나누어 봐도 '이수학점이 높은 학생의 GPA도 높고, 탈락 사례는 낮은 학점·GPA 쪽에 있다'는 비슷한 모습이 보입니다.
다만 이 장의 목적은 **"두 학과가 비슷하다"는 결론 자체보다, 전체에서 보인 관계가 학과 안에서도 유지되는지 확인하는 것**입니다.

“지금까지는 확보한 자료에서 어떤 특징과 관계가 나타나는지
탐색했습니다. 이제 그 결과를 조사하지 않은 전체나 미래 집단까지
확대해 해석하려면, 결과가 얼마나 달라질 수 있는지 고려해야 합니다.

-> 이를 위해 확률모형과 표집분포를 확인

관계 탐색 결과를 문장으로 정리하기

관찰된 차이는 후속 질문의 출발점이다

관찰된 사실

가상 자료에서 탈락 집단의 평균 GPA는 1.95, 비탈락 집단은 3.22였다.

가능한 설명

학업 참여 차이 또는 집단 구성 차이와 관련될 수 있다.

한계와 다음 질문

유효 n은 각각 2명·5명이다. 같은 조건의 더 많은 자료에서도 유지되는가?

여기서 “표본의 차이를 어디까지 믿을 것인가?”라는 질문이 생긴다

중도탈락 현황 분석의 대상과 기준

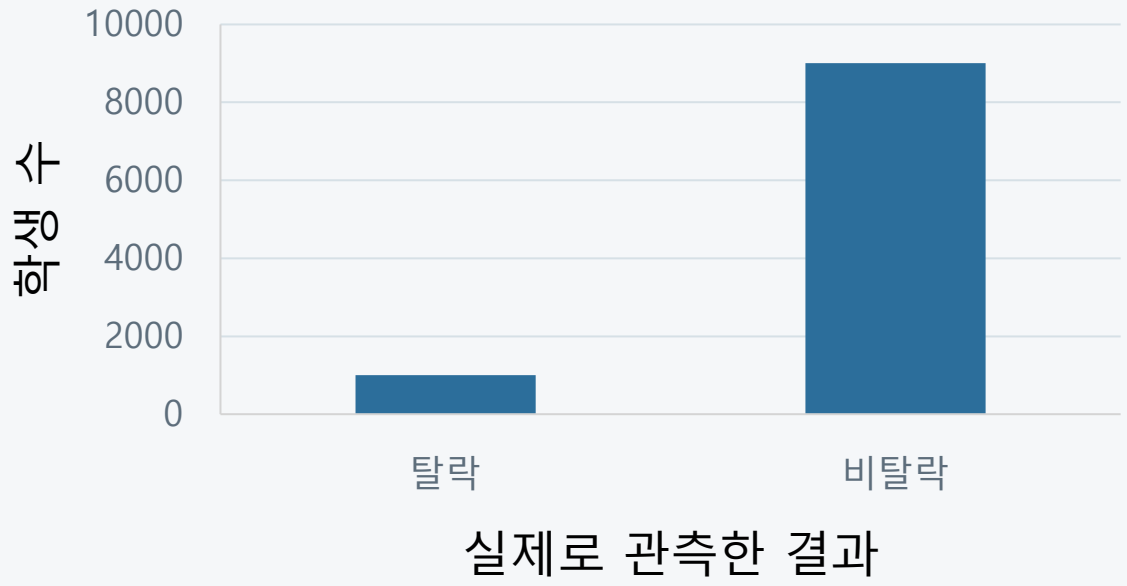
가상 사례: 최근 10개 입학연도의 전체 학생 10,000명, 입학 후 1년 이내 탈락 1,000명

분석 기준	이 예제의 정의
대상	10개 입학연도 학생 전체 10,000명
결과	입학 후 1년 이내 탈락=1, 그 외=0
관찰기간	학생마다 1년 확보. 입학연도가 달라도 같은 기간 비교
자료 범위	탈락 1,000명과 비탈락 9,000명을 모두 포함
현황 지표	1년 이내 탈락률 = 1,000 / 10,000 = 10%

탈락 학생만 모은 자료로는 전체 탈락률을 구할 수 없다. 같은 기준의 분모가 필요하다

관측분포: 확보한 자료의 현황

첫 질문: 실제 10,000명에게 어떤 결과가 있었는가?



실제로 관측한 결과

관측분포

탈락 10%, 비탈락 90%
이미 확보한 학생들의 결과를 요약한다.

현황 분석의 확장

입학연도·학과별 건수와 비율,
탈락 여부별 GPA 분포 등을 비교한다.

여기서 말할 수 있는 것

대상 전체 10,000명의 과거 탈락률은 10%.
이 수치를 보고하는 데 검정은 필수가 아니다.

다음 질문이 전체 추정이나 미래 변동으로 넓어질 때 확률모형을 도입한다

확률분포 도입

확률모형: 다른 집단의 결과를 계산하는 가정

두 번째 질문: 비슷한 조건의 학생 100명에서는 탈락자가 몇 명 나올까?

출발점

과거 관측률은 10%였다. 이를 비교 기준 p=0.10으로 놓는 설명용 모형을 세운다.

학생 한 명의 결과

탈락=1, 비탈락=0. 탈락확률 10%, 비탈락확률 90%의 베르누이 모형이다.

100명의 탈락자 수

학생별 확률이 같고 서로 독립이라고 가정하면 K는 이항분포를 따른다.

이 모형의 역할

평균적으로 10명을 예상하지만, 8명·12명·15명 등의 가능성도 계산한다.

과거 10%를 미래에 적용하려면 학생 구성·제도 변화를 검토한다. 과거 비율만으로 미래를 확정하지 않는다

확률분포 도입

확률모형: 다른 집단의 결과를 계산하는 가정

베르누이 모형 : 한 번의 시행에서 결과가 두 가지(탈락/비탈락) 중 하나로 나타나는 확률모형

이항분포는 이런 베르누이 시행을 여러 명에게 반복했을 때 탈락자가 총 몇 명 나오는가를 보는 것입니다.

정확히 k명이 탈락할 확률은 다음 공식으로 계산합니다.

$$P(K = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

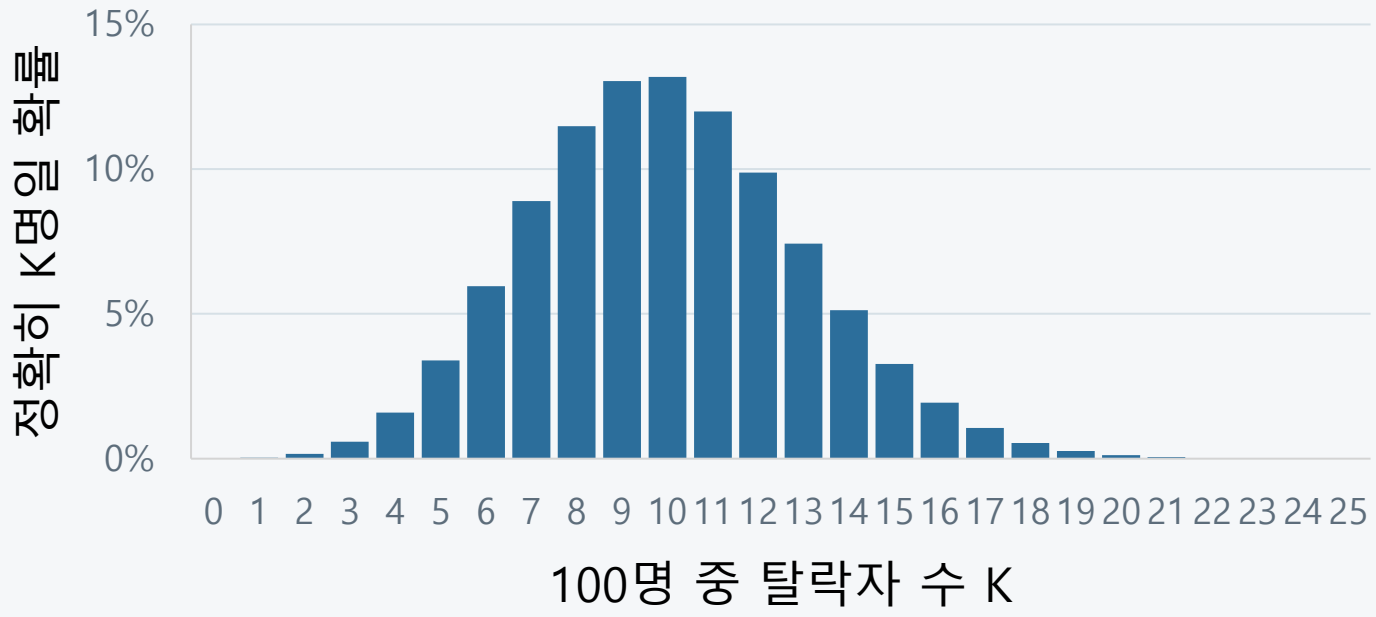
$k = 10$을 넣으면:

$$P(K = 10) = \binom{100}{10} \times 0.1^{10} \times 0.9^{90} \approx 0.131865 = \boxed{13.19\%}$$

각 부분의 의미는 다음과 같습니다.

이항분포 그래프의 가로축과 세로축

가정: 학생별 탈락확률 10%. 그래프는 100명 중 정확히 K명이 탈락할 확률을 표시한다



가로축 K

100명 중 탈락자 수
0명, 1명, 2명, ..., 100명

세로축

각 건수가 나올 확률
K=10 막대 높이 약 13.19%

서로 다른 확률

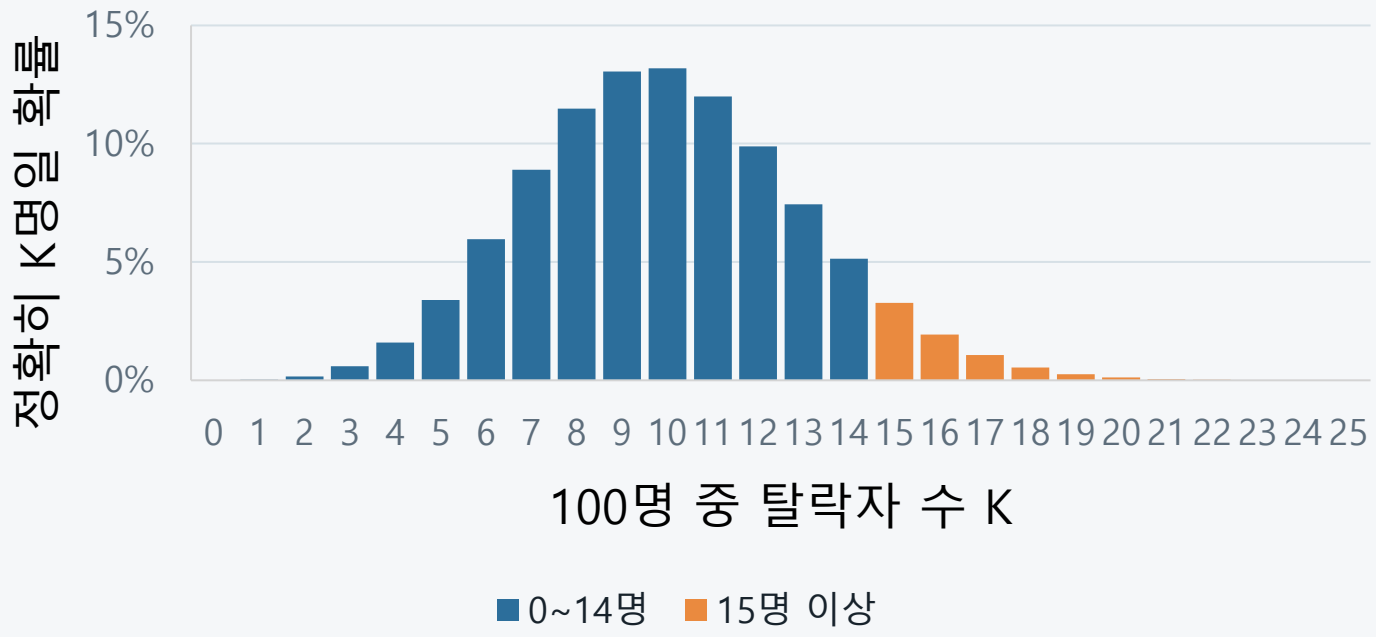
한 학생의 탈락확률은 10%.
100명 중 정확히 10명일 확률은 13.19%.

자료에서 센 히스토그램과 구분한다. 이 그림은 가정으로 계산한 확률이며 0~25명만 표시했다

확률분포

관측한 15명을 기준 확률과 비교

세 번째 질문: 실제 15명이 탈락했다면, 기준 p=10%에서도 이 정도가 얼마나 가능한가?



주황색 막대

15명, 16명, 17명, ...
관측한 15명 이상에 해당한다.

확률을 더하면

P(K ≥ 15) ≈ 7.26%
표시 밖의 26~100명도 계산에 포함한다.

해석 문장

기준 모형하에서 100명을 관찰할 때
15명 이상 나올 가능성은 약 7.26%다.

7.26%는 기준 모형이 참일 확률이 아니다. 기준 가정에서 관측 결과의 드문 정도를 평가한다

확률분포

표집분포: 표본 탈락률의 흔들림

전체 자료를 얻기 어려워 100명만 무작위 조사한다면, 뽑힌 학생에 따라 비율이 달라진다

설명용 반복 조사	뽑힌 학생 수	탈락자 수	표본 탈락률
첫 번째	100	8	8%
두 번째	100	12	12%
세 번째	100	9	9%
네 번째	100	15	15%

표집분포는 이렇게 반복해서 계산한 "표본 탈락률"의 분포다.
실제로 여러 번 조사해야 하는 것은 아니다. 이론이나 재표집으로 변동을 평가한다.
표본의 15%로 전체를 추정할 때 신뢰구간과 검정의 근거로 사용한다.

"다른 100명을 뽑으면 탈락 학생이 8명이나 12명일 수도 있습니다. **이처럼 누구를 뽑느냐에 따라 표본 탈락률이 8%, 12%, 15% 등으로 달라지는 것**을 설명하려는 예제입니다.

불확실성은 보통 **표준오차(Standard Error)** 와 **신뢰구간(Confidence Interval)** 으로 평가합니다.

비율의 표준오차는 대략:

$$SE = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

입니다.

예를 들어:

$$\hat{p} = 0.10, \quad n = 100$$

이면:

$$SE = \sqrt{\frac{0.1 \times 0.9}{100}} = \sqrt{0.0009} = 0.03$$

즉 약 3%p 정도 흔들릴 수 있다고 볼 수 있습니다.

$$P(-1.96 \leq Z \leq 1.96) \approx 0.95$$

그다음 95% 신뢰구간을 단순 근사하면:

$$0.10 \pm 1.96 \times 0.03$$

$$\boxed{\text{추정값} \pm 1.96 \times \text{표준오차}}$$

= 4.1% ~ 15.9%

표본에서는 탈락률이 10%로 관찰되었지만,
표본을 다시 뽑을 때 생기는 변동을 고려하면 모집단 탈락률은 대략 4~16% 범위일 가능성이 있다.

관측분포·확률모형·표집분포의 사용 시점

세 개념은 분석 질문이 현황 설명에서 추정으로 넓어질 때 연결된다

질문	사용하는 개념	이 예제에서 남길 결과
실제로 무슨 일이 있었나?	관측분포	10,000명 중 탈락 1,000명, 관측률 10%
가정하에서 어떤 결과가 가능한가?	확률모형	학생 100명의 가능한 탈락자 수와 확률
일부로 계산한 비율은 얼마나 흔들리나?	표집분포	표본 탈락률의 변동, 신뢰구간·검정

현황을 보고하는 목적이라면 관측분포로 충분할 수 있다. 추정하려는 대상과 질문을 먼저 정한다

확률분포에서 무엇의 확률을 구하는가

분포는 가능한 결과에 확률을 부여한다. 질문에 맞는 사건과 가정을 먼저 정한다

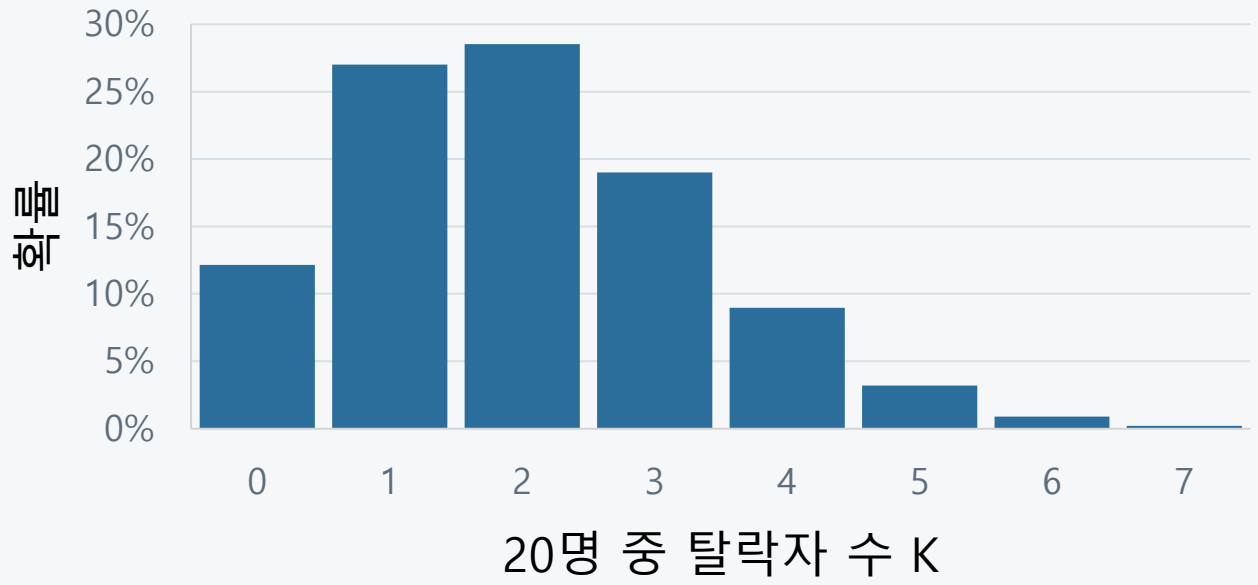
질문	분포를 적용할 대상	계산하는 확률
20명 중 정확히 2명이 탈락할까?	탈락자 수 K	이항: P(K=2)
하루 상담이 5건 이상 들어올까?	하루 상담 건수 N	포아송: P(N≥5)
처리 시간이 10~15분일까?	연속형 처리 시간 T	적합한 연속분포: P(10≤T≤15) - 정규분포, 감마분포, 지수분포
평균 차이가 이만큼 클까?	평균 차이의 검정통계량	귀무가설하의 꼬리확률

연속분포에서는 정확히 한 점의 확률은 0이고 구간의 면적이 확률이다.
히스토그램만 보고 분포를 지정하지 않는다. 가능한 값·생성과정·적합성을 함께 확인한다.

귀무가설 하의 꼬리확률은, 귀무가설이 맞다고 가정했을 때 현재 관측 결과만큼 극단적인 값이 나올 확률이며, 이를 p-value라고 한다.

이항분포는 고정된 n번 중 발생한 횟수

20명 각각의 탈락확률을 10%로 가정할 때, 탈락자 수 K의 확률을 계산한다



필요한 조건

n=20 고정, 각 결과 0/1
독립 시행, 같은 p=0.10

가능한 값

K = 0, 1, …, 20
평균 np=2, 분산 np(1−p)=1.8

정확히 2명일 확률

P(K=2) ≈ 28.52%
stats.binom.pmf(2, 20, .1)

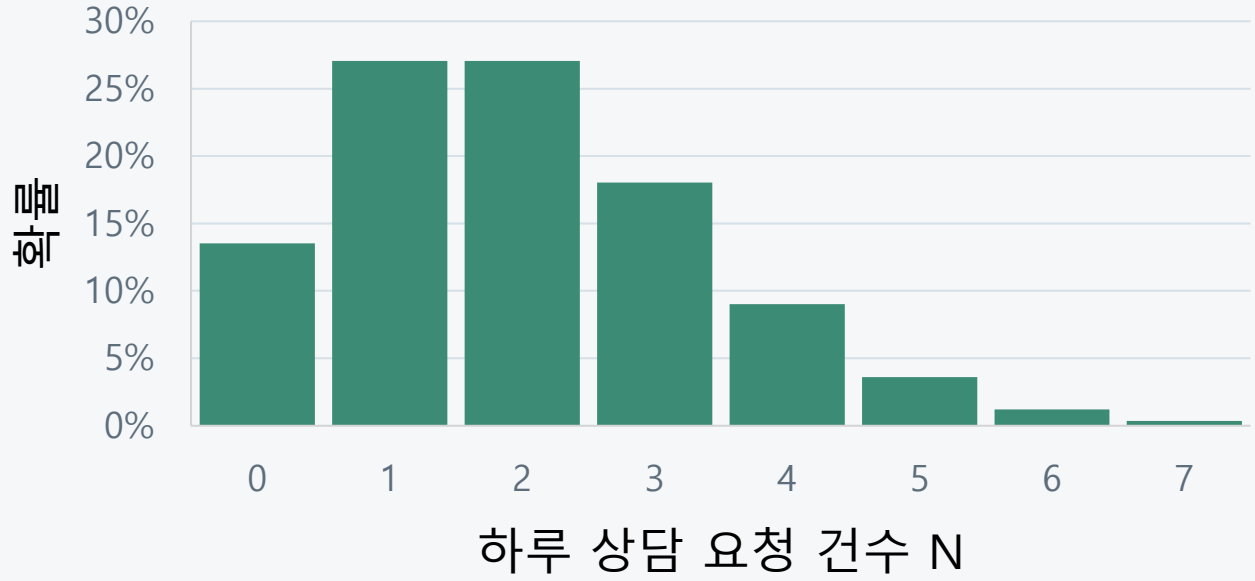
P(K=2) = 190 × 0.1^2 × 0.9^{18}. 그래프는 0~7만 표시했고 실제 범위는 0~20이다

확률분포

포아송분포는 일정 구간의 사건 횟수

포아송분포(Poisson distribution)는 일정한 시간이나 공간 안에서 어떤 사건이 몇 번 발생하는지를 나타내는 확률분포입니다.

하루 평균 상담 요청이 2건일 때 5건 이상 몰리는 날의 가능성을 계산한다



필요한 조건

같은 길이의 기간, 안정적인 발생률
겹치지 않는 구간의 독립성

평균과 분산

λ=2, 평균=분산=2
가능한 값은 0,1,2,...

5건 이상일 확률

1 − P(N≤4) ≈ 5.27%
stats.poisson.sf(4, 2)

20명 중 몇 명인지는 이항, 하루에 몇 건 들어오는지는 포아송. 몰림·과산포가 있으면 재검토

확률분포

1. 탈락/비탈락 집단의 GPA 분포 확인

   → 히스토그램, 상자그림, Q-Q plot → 왜도와 이상치 확인

2. 표본 수와 분포 특성 확인 → 정규성 가정이 크게 문제되지 않는가?

   → 이상치가 심하지 않은가? → 두 집단의 분산은 비슷한가?

3. 검정 방법 선택

  → 독립 두 집단 평균 비교: Welch t-test → 분포가 매우 비정규적이거나 이상치가 심함: Mann–Whitney U 검토

정규분포와 표준화 점수

평균은 중심, 표준편차는 퍼짐을 결정한다



표준화

z = (x − 평균) / 표준편차

정규분포일 때

평균 ± 1.96σ에 약 95%

주의

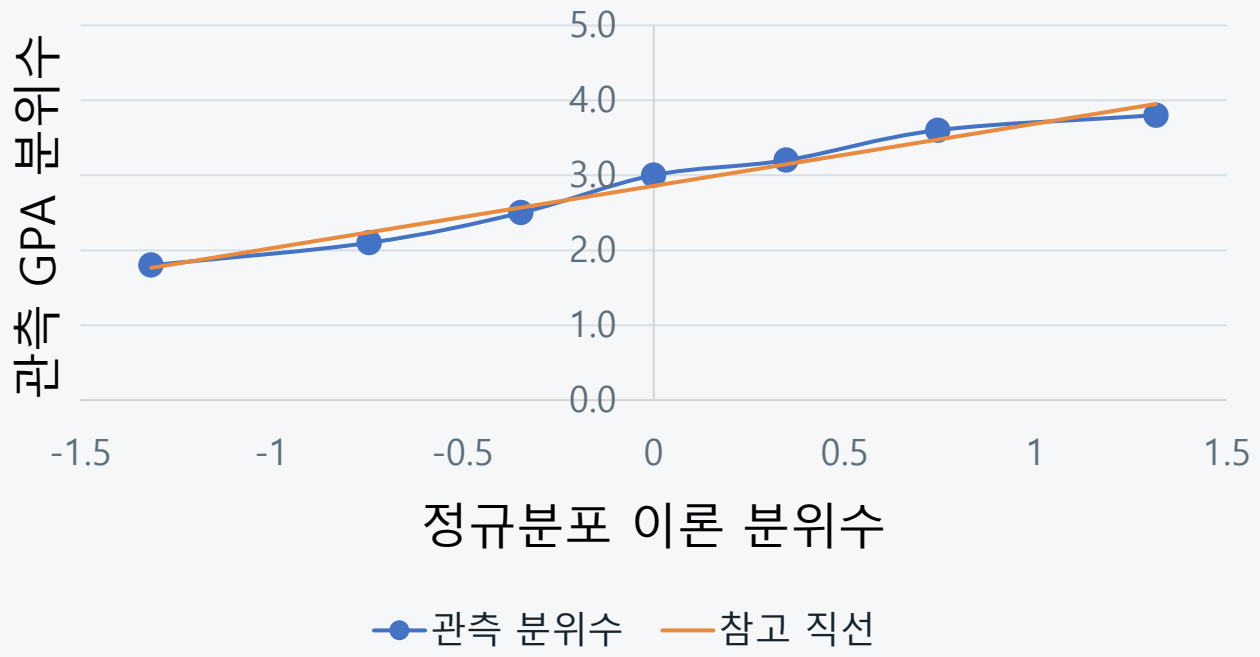
곡선의 높이가 곧 확률은 아니다
-> 평균 근처의 값들이 많이 나타남

표준화는 척도를 바꾸며, 분포를 정규분포로 바꾸지는 않는다

확률분포

Q–Q plot으로 분포 가정 확인하기

이론분포의 분위수와 실제 자료의 분위수를 비교한다



직선에 가까움

분포 모양이 대체로 유사

곡선·꼬리 이탈

왜도 또는 꼬리 차이 가능

함께 볼 것

히스토그램·이상치·표본 수

-> 정규분포

stats.probplot(x, dist="norm", plot=plt)

확률분포 · 가정 점검

Q–Q plot으로 분포 가정 확인하기

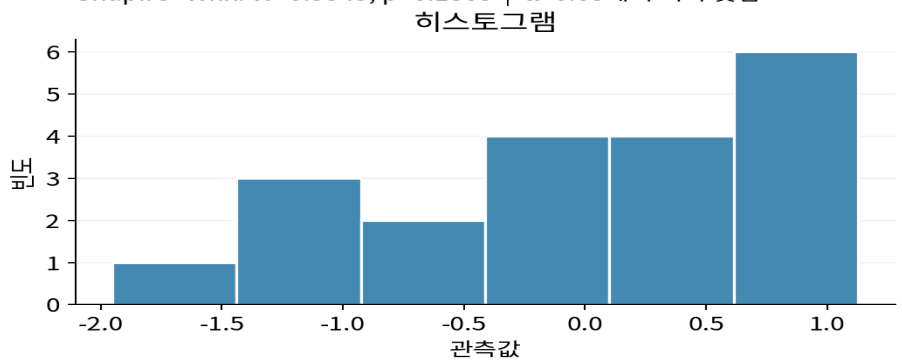
정규분포 :
히스토그램은 대체로 대칭적이고,
Q-Q plot 점들은 직선에 가깝고,
Shapiro-Wilk의 p-value가 0.05보다 크다

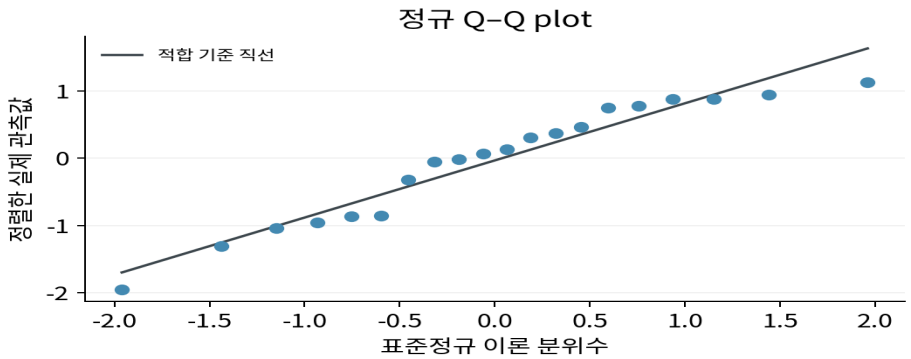
생성 수식이 다른 두 분포: 히스토그램과 정규 Q–Q plot

동일한 Z 20개 사용 · Z ~ N(0, 1), seed=42 · 설명용으로 선택한 가상 예제

A. 정규분포 X = Z

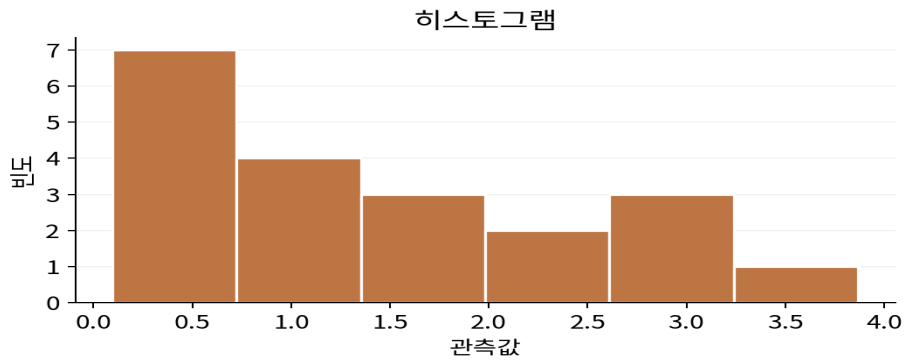
Shapiro–Wilk: W=0.9343, p=0.1868 | α=0.05에서 기각 못함

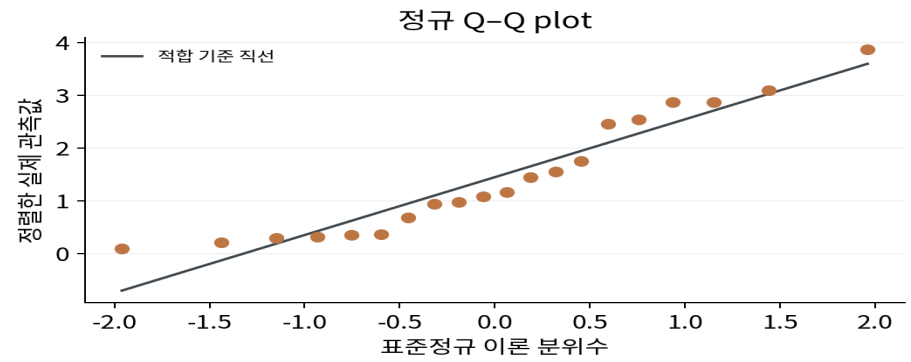




B. 로그정규분포 Y = exp(1.2Z)

Shapiro–Wilk: W=0.9096, p=0.0626 | α=0.05에서 기각 못함





Q–Q 좌표: (norm.ppf((i−0.5)/20), i번째 정렬값). 정규분포와 비슷하면 대체로 직선을 따른다.

생성 분포와 작은 표본의 모양은 구분해야 한다. p ≥ 0.05도, 그래프도 정규성을 증명하지 않는다.

표준편차와 표준오차의 차이

학생들의 차이와 평균 추정의 불확실성은 서로 다르다

개념	무엇이 변하는가	예제
표준편차 SD	학생들 사이의 GPA	두 표본에서 모두 0.60
표준오차 SE	다른 표본을 뽑았을 때 평균	n=25: 0.12 / n=100: 0.06

평균의 표준오차: SE ≈ s / √n : s = 표본 데이터의 표준편차

독립성 등 조건이 적절할 때 사용하는 근사이다.

표본 수가 4배이면 SE는 약 절반

하지만 표본 편향이나 측정 오류까지 사라지는 것은 아니다.

통계적 검정이 필요한 이유

표본에서 차이가 보인다고 모집단에도 차이가 있다고 단정할 수 없다

EDA에서의 관찰

A집단과 B집단의 평균 GPA가 다르게 보인다.

검정에서의 질문

모집단 평균이 같아도, 이 정도 이상의 표본 차이가 생길 수 있는가?

함께 보고할 것

평균 차이의 크기와 신뢰구간, 표본 수, 가정 및 연구 설계

검정은 일반화의 근거를 보완한다. 인과관계는 연구 설계까지 필요하다

귀무가설·p값·유의수준

p값은 “차이가 없는 모형과 데이터가 얼마나 양립하는가”를 살핀다

용어	의미
귀무가설 H_0	비교할 모집단 평균 차이는 0이다
p값	H_0와 가정하에서 관측만큼 또는 더 극단적인 결과가 나올 확률
유의수준 α	자료를 보기 전에 정하는 기각 기준. 예: 0.05
$p \geq 0.05$	차이의 증거가 부족함. 같다는 증명은 아님

p=0.03을 “귀무가설이 참일 확률 3%”로 읽으면 안 된다

$p < 0.05$ → 차이가 있다고 볼 근거 있음 → 귀무가설 기각
$p \geq 0.05$ → 차이가 있다고 볼 근거 부족 → 귀무가설 기각 못함

검정 방법을 선택하기 전에 정할 것

비교할 양과 독립·대응 구조를 정한 뒤 각 방법의 원리를 학습한다

질문	자료 구조	배울 방법
두 집단의 평균이 다른가?여기에 수식을 입력하십시오.	서로 다른 학생의 수치값	Welch t 검정
같은 학생의 평균이 변했나?	개인별 전후 쌍	대응 t 검정
세 집단 이상의 평균이 다른가?	독립 집단의 수치값	ANOVA / Welch ANOVA
두 범주가 연관되는가?	관측 빈도 교차표	카이제곱 / Fisher
한 집단의 값이 더 큰 경향인가?	독립 두 집단의 순위	Mann–Whitney U

검정의 귀무가설은 방법마다 다르다. “차이 검정”이라는 이름만으로 바꾸어 쓰지 않는다

종합 실습의 분석 순서

질문: 탈락 여부에 따라 학업 특성이 어떻게 다른가?

순서	실습 활동	제출 결과
1	GPA·학점의 단변량 요약	유효 n·대표값·산포·그래프
2	탈락 여부별 GPA 비교	집단별 n·평균·중앙값·분포
3	학과별 조건부 비교	분모가 명시된 교차표
4	일반화 가능성 검토	표본 크기·관측 시점·가정·추가 자료

질문별로 다시 고르는 분석 도구

이번 강의에서 배운 도구를 어떤 질문에 쓰는지 연결한다

질문	도구	남길 결과
값의 수준과 퍼짐은?	평균·중앙값·IQR·히스토그램	대표값과 분포 해석
기준 이하 비율은?	ECDF	건수·분모·누적비율
두 변수와 여러 변수의 관계는?	산점도·교차표·히트맵·PCA	조건부 비교와 구조 요약
관측한 차이의 불확실성은?	신뢰구간·목적에 맞는 검정	효과 크기·구간·가정
척도를 바꿀 필요가 있나?	로그·거듭제곱 변환·표준화	선택 근거와 전후 결과

도구 이름을 나열하기보다 질문 → 계산 → 그래프 → 해석의 순서로 설명한다

도구 정리

Python 도구별 역할

실행 환경, 데이터 처리, 시각화, 통계 계산을 구분한다

도구	역할	이 강의의 사용 예
Colab / Jupyter	코드·결과·설명을 같은 노트북에 기록	순서대로 실행하고 해석 작성
pandas	표를 정리하고 집계	describe, groupby, crosstab
seaborn / matplotlib	통계 그래프와 축·범례 조정	histplot, boxplot, scatterplot
SciPy	확률분포·통계 검정	probplot, ttest_ind, chi2_contingency
scikit-learn / statsmodels	전처리·차원축소 / 추론 모형	PowerTransformer·PCA / 회귀

자동 보고서의 숫자도 변수 정의·분모·가정이 맞는지 직접 확인한다

핵심 정리와 참고 자료

EDA의 결과는 그래프 목록이 아니라 근거와 한계가 있는 설명이다

분포와 관계

변수 유형에 맞는 그래프를 고르고 n·분모·단위를 함께 읽는다.

확률분포와 검정

표본 변동을 고려하되, 효과 크기·가정·연구 설계까지 해석한다.

분포 변환

분석 목적을 먼저 정하고 변환 전후 결과와 원래 단위를 확인한다.

참고: NIST EDA Handbook, seaborn 시각화 문서, SciPy 통계 API,
scikit-learn 전처리·PCA 문서 및 첨부 Word 2종